Clase 19 — Cómo garantiza TCP la fiabilidad

Bloque: C (Transporte) · **Semana:** 7 · **Duración estimada:** 110 min **Correlativa:** Clase 18 (TCP: establecer y cerrar la conexión) · **Sigue con:** Clase 20 (control de flujo vs. control de congestión) **Material de referencia:** Tanenbaum cap. 6 (§6.5 "Reliable Transmission") · RFC 9293, RFC 2018, RFC 7323, RFC 5681, RFC 6928

🎯 Al terminar esta clase vas a poder

1. Traducir cada palabra del contrato de TCP (*entrega ordenada, sin duplicados, sin errores*) al mecanismo que la implementa.
2. Explicar por qué los números de secuencia son **imprescindibles** aunque el checksum ya detecte errores, y por qué el ACK es **acumulativo y numera bytes**.
3. Calcular un **RTO** con Jacobson/Karels a partir de una serie de muestras de RTT, y aplicar el **backoff exponencial**.
4. Calcular el **BDP** de un enlace y deducir el **tamaño mínimo de ventana** para no desperdiciar capacidad.
5. Dibujar la **ventana deslizante** con `LastByteAcked`, `LastByteSent` y los bytes en vuelo.
6. Describir cómo **SACK**, **timestamps** y **PAWS** cambian la recuperación de una pérdida, y por qué el flag **PSH no garantiza** un mensaje por `read()` en el destino.

> **Prerequisito del examen:** `2011-03 P4` (10 pts — el diagrama de 4 segmentos), `2010-12 P4` (8 pts — números de secuencia y timers), `2009-12 P2` (10 pts — fiabilidad en transporte vs. enlace), `2001-12 Problema 2` (PUSH y segmentación).

PARTE 1 — TEORÍA

1.1 El contrato de TCP, palabra por palabra

Lo que promete	Mecanismo que lo cumple
Ordenada	**Números de secuencia** + buffer de reordenamiento en el receptor
Sin duplicados	el receptor **descarta** lo que ya tiene: `rcv_nxt` es el puntero que avanza
Sin errores	**checksum** (que detecta) + **retransmisión** (que corrige)

La distinción "detectar" vs. "corregir" se pregunta siempre: el checksum de TCP **no corrige nada**, solo descarta el segmento. Lo que repara es la **retransmisión**, que necesita saber *qué* falta. **Orden → duplicados → errores** es una cadena de dependencias: no podés evitar duplicados si no sabés qué tenés, y no podés retransmitir si no numerás.

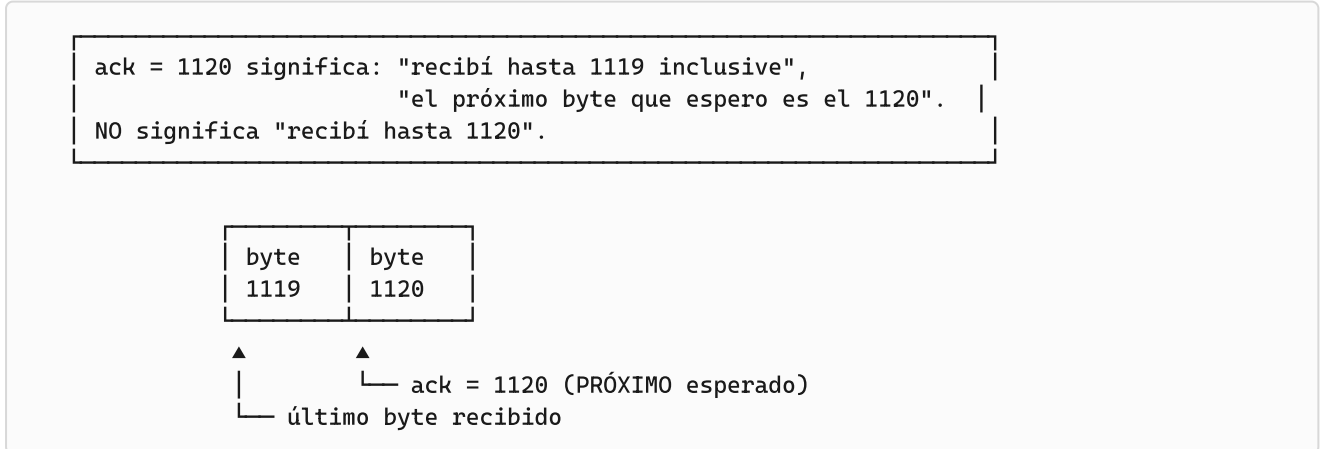
1.2 Números de secuencia: por qué hacen falta

La respuesta oficial del examen 2010-12 (P4a) es cortita y hay que saberla de memoria:

> TCP es **pipelined**: el emisor envía varios paquetes antes de recibir confirmación de alguno de ellos. Por esta razón, el emisor debe poder distinguir **a qué paquete o paquetes corresponde una determinada confirmación**, por lo que numera los paquetes enviados secuencialmente.

Hay un segundo motivo, más profundo: **el checksum no te dice *qué* falta**. Te dice "este paquete vino mal". Para retransmitir el correcto necesitás un identificador que sobreviva al viaje, y sin números tampoco podés evitar duplicados, porque no sabés qué tenés ya.

TCP numera BYTES, no segmentos. El `seq` de un segmento es el número de su **primer byte**, y el `ack` es el **próximo byte esperado**, no el último recibido:



Por eso los ACKs son **acumulativos** (*cumulative ACK*): un solo ACK puede confirmar varios segmentos. Si B recibe S1..S4 seguidos, un `ack=1180` confirma los cuatro.

1.3 Timers: la pregunta de 2010-12 (P4b), resuelta

La situación. A mandó un segmento. Pasó un rato. No llegó el ACK. ¿Por qué?

```
┌──────────────────────────────────────────────────────────────┐
│  ¿Por qué no llegó el ACK?                                   │
│   (1) Se perdió el SEGMENTO de datos                         │
│   (2) Se perdió el ACK                                       │
│   (3) El ACK no se perdió: simplemente TARDÓ                 │
└──────────────────────────────────────────────────────────────┘
                 │
                 ▼
¿Qué puede observar A?  En los tres casos, EXACTAMENTE LO MISMO: "no sé".
```

Las tres tienen **la misma consecuencia observable**: la ausencia del ACK. No puede retransmitir "por las dudas" al instante (sería un duplicado puro) ni esperar indefinidamente (si el segmento se perdió de verdad, la conexión se cuelga). **La única salida es retransmitir**, y para eso hay que decidir **cuándo**: esperar un "tiempo prudencial". Eso es un **timer**.

El enunciado oficial dice textualmente "*la única solución es retransmitir el paquete no confirmado… necesita un timer que le indique cuándo hacerlo*" (la cita completa y el desglose de los 4 puntos están en §3.1).

Un detalle que suma puntos: el timer se arma **una vez por segmento enviado**, no una vez por conexión. Si A manda S1..S4 seguidos hay **cuatro timers corriendo**: es la misma razón por la que hacen falta los números de secuencia.

1.4 El RTO y cómo se estima (Jacobson/Karels)

El **RTO** (*Retransmission Timeout*) es cuánto espera TCP antes de declarar perdido un segmento. Si fuera fijo: demasiado corto → retransmitís de más y generás **duplicados**; demasiado largo → cada pérdida cuesta un RTO entero de latencia. La solución: **medir**.

```
SRTT   = suavizado del RTT
RTTVAR = suavizado de la variación del RTT
RTO    = SRTT + 4 · RTTVAR

Con cada nueva muestra R:
  RTTVAR ← (1 − β) · RTTVAR + β · |SRTT − R|      ← con el SRTT ANTERIOR
  SRTT   ← (1 − α) · SRTT   + α · R
  RTO    ← SRTT + 4 · RTTVAR

α = 7/8   (recuerdo alto: el pasado pesa más que el presente)
β = 1/4   (reacción rápida a los cambios)

Primera muestra:  SRTT = R ;  RTTVAR = R/2 ;  RTO = R + 4·(R/2) = 3R
```

¿Por qué `SRTT + 4·RTTVAR` y no un factor fijo? Porque la variación es la **incertidumbre** de la medida. Si el RTT está en 100 ms con variación de ±20 ms, un RTO de `100 + 4·20 = 180 ms` deja pasar casi cualquier medición posible. Además hay **un piso y un techo**: el RTO no puede ser menor que un mínimo histórico (1 s) ni mayor que un máximo (60 s).

Un detalle fino: no se toman muestras de RTT de los segmentos retransmitidos (algoritmo de **Karn**): si retransmitiste S, no sabés si el ACK que llegó corresponde al S original o a la retransmisión, y si es a la retransmisión el RTT medido incluye el RTO entero y **infla** la estimación. Los **timestamps** lo resuelven: el ACK hace eco del `TSval` del segmento.

Backoff exponencial

Cuando un timeout se dispara, el RTO **se duplica**: con `RTO = 240 ms`, los timeouts sucesivos dan 480 ms, 960 ms, 1920 ms. **¿Por qué?** Un timeout **no** es evidencia de congestión: puede ser un blackout o un enlace caído. Si además de subir el RTO bajás la ventana, lo que decís es "esto parece una avería, esperemos más". Ojo: **las dos cosas pasan a la vez** y no se contradicen — el `cwnd` protege a la red, el backoff protege al emisor de insistir contra algo que no responde.

1.5 La ventana deslizante

En *stop-and-wait* el emisor manda un segmento y **espera**: el enlace queda ocioso la mitad del tiempo. TCP no: manda hasta `ventana` bytes sin esperar confirmación.

```
El emisor A, con ventana = 8 bytes (4 segmentos de 2 bytes):

LastByteAcked           LastByteSent         la ventana
      |                       |                   |
------|-----------------------|-------------------|-----
   100                      108                 110
bytes ya confirmados     último byte       no se puede
                         enviado          pasar de acá

bytes en vuelo = LastByteSent - LastByteAcked = 108 - 100 = 8
(la ventana es justamente 8: la ventana está LLENA)
```

`LastByteAcked` = último byte confirmado · `LastByteSent` = último byte enviado · `LastByteAcked + ventana` = el tope que el emisor no puede pasar.

Además el emisor no puede mandar más allá de lo que el receptor dice que entra (el `rwnd`), y por eso `ventana efectiva = min(cwnd, rwnd)`. → El mínimo y el `cwnd` están desarrollados en las Clases 20 y 21; lo que hay que internalizar acá es el **sentido físico**: la ventana es "cuánta información puedo dejar en el camino antes de tener que parar a esperar".

1.6 Por qué la ventana tiene que ser ≥ BDP

BDP = *Bandwidth-Delay Product*: los bytes que están "en vuelo" en un momento dado si el emisor llena el enlace sin pausa.

```
BDP = ancho_de_banda × RTT          [bytes] = [bits/s] × [s] / 8
```

El argumento, en una frase: si la ventana es menor que el BDP, **el enlace no está lleno**, y estás pagando un enlace de 100 Mbps para usar 4 Mbps. Con 100 Mbps y RTT 80 ms:

```
BDP = 100 · 10⁶ bits/s × 0,080 s = 8 · 10⁶ bits = 1 000 000 bytes ≈ 1 MB

ventana 64 KB:  throughput = 65 536 / 0,080 = 819 200 B/s = 6,55 Mbit/s   ← 15× menos
ventana  1 MB:  throughput = 1 048 576 / 0,080 = 13 107 200 B/s = 104,9 Mbit/s ← lleno

En MSS (MTU 1500 − 20 IP − 20 TCP = 1460 B):  1 000 000 / 1460 ≈ 685 segmentos
```

Ésta es la razón por la que el `rwnd` importa tanto y por la que existe el *window scale*: el campo `window` de la cabecera son **16 bits** (máximo 65 535 bytes), y con RTT de 80 ms eso limita el throughput a 6,55 Mbit/s. Con *Window Scale* (`RFC 7323`, factor negociado en el SYN) se llega a megabits.

1.7 Recuperación rápida: fast retransmit y SACK

Fast retransmit está desarrollado en la Clase 21 (§1.4c): 3 ACKs duplicados → retransmitir sin esperar el RTO. La idea: **los ACKs duplicados son la señal de que los datos posteriores llegaron**. Si hubiera un problema grave de red no habrían llegado.

SACK (`RFC 2018`)

El problema sin SACK. El ACK acumulativo es todo o nada:

```
A manda S1(1000), S2(1046), S3(1092), S4(1138). Se pierde S2.
B recibe S1, S3, S4.
B → A:  ACK=1046  ACK=1046  ACK=1046      (3 duplicados)
A sólo sabe: "me falta algo a partir de 1046".
A NO sabe si es S2 solo, o S2+S3, o S2+S3+S4.
→ TCP clásico (Tahoe): retransmite S2, espera, y recién con el ACK
  siguiente descubre que S3 también faltaba. UNO POR UNO.
```

Con SACK, el receptor manda bloques extra que dicen *qué* bloques exactos recibió. Cada bloque es un par `(borde izquierdo, borde derecho)` en números de secuencia absolutos, y van en el mismo segmento que el ACK acumulativo:

```
B → A:  ACK=1046  SACK=[1092..1138]      (recibí S3)
B → A:  ACK=1046  SACK=[1092..1184]      (recibí S3 y S4)
```

A ahora sabe que S3 y S4 están bien, así que **retransmite solo S2**, y cuando S2 llega todo se confirma de un golpe. **SACK no cambia la corrección: cambia cuánto tarda.**

⚠️ **Detalle que se pregunta:** SACK se **negocia en el SYN** (opción *SACK permitted*). Si no se negocia, no hay bloques SACK. Y como SACK solo informa de bytes **dentro de la ventana**, no reemplaza al ACK acumulativo: siempre viene acompañado.

Timestamps y PAWS (`RFC 7323`)

Un **timestamp** (32 bits, reloj del emisor) que el receptor devuelve **ecoado** en el ACK, con tres usos: **medir el RTT sin ambigüedad** (resuelve el problema de Karn), **detectar duplicados viejos** —si un segmento trae un `TSval` menor que el último visto, es un resto de una sesión anterior y se descarta, que es

exactamente lo que hace **PAWS** (*Protect Against Wrapped Sequence numbers*)— y **proteger del wrap-around** del número de 32 bits. **PAWS es obligatorio si ambos extremos activaron timestamps.**

1.8 Por qué el flag PSH no garantiza nada (examen 2001-12, Problema 2)

> **Es sabido que, al enviar mensajes mediante una conexión TCP, el mero uso del bit de PUSH no asegura que, en el destino, la capa de aplicación reciba un mensaje por vez.**

Qué es PSH. Es una **sugerencia** de "no tengo más datos en el buffer, podés entregar lo que tengas". Cuando un segmento llega con PSH, el receptor entrega a la aplicación todo lo acumulado en el buffer **en un solo `read()`**:

```
A: write("hola") → write("chau")     ← dos mensajes de la aplicación
┌──────────────────┬──────────────────┐
│ "hola"  PSH=1    │ "chau"  PSH=1    │   ← dos segmentos
└──────────────────┴──────────────────┘
B: read() → "hola" / read() → "chau"   ← lo que uno ESPERA
B: read() → "holachu"                   ← lo que REALMENTE PASA
```

Por qué no funciona — tres razones acumulativas:

1. **PSH no transporta información de frontera de mensaje.** El campo es **un bit**, y no hay ningún campo que diga "este `read()` debe terminar acá". TCP es un **flujo de bytes** (*byte stream*) por diseño: tiene bytes, no mensajes.
2. **Los límites de segmento son invisibles para la aplicación.** El receptor ve una cola de bytes. Si los dos segmentos con PSH llegan en orden inverso, cuando llega el primero el buffer queda `["hola","chau"]` **contiguo**, y un solo `read()` devuelve todo junto.
3. **La segmentación.** Si un mensaje es más grande que el MSS queda partido en varios segmentos, y el PSH final llega después de bytes ya almacenados.

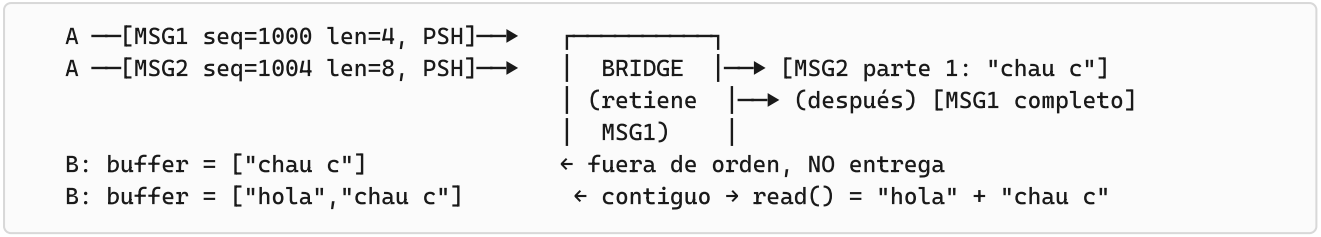
El laboratorio que pide el enunciado

El enunciado pide **construir un bridge** que manipule orden y segmentación: es factible, un bridge es software y el reordenamiento se hace con un **buffer de retardo**.

(a) Arribo simultáneo de dos mensajes — el bridge retiene el segmento del mensaje 1:

```
A ──[MSG1 seq=1000 len=4, PSH]──►   ┌──────────┐
                                    │  BRIDGE  │ retiene MSG1
A ──[MSG2 seq=1004 len=4, PSH]──►   │(reordena)│──► MSG2 primero
B: buffer = ["chau"]          ← PSH de MSG2, pero falta MSG1 → NO entrega
B: buffer = ["hola","chau"]   ← llega MSG1: contiguo
B: read() → "holachu"         ← UN read(), dos mensajes  ✗
```

(b) Un mensaje y parte del siguiente — además, el bridge **parte** el segundo mensaje y deja pasar primero su primera mitad:



⚠️ **El detalle que hace el experimento correcto:** los `seq` deben ser **coherentes y contiguos** (`MSG2` con `seq = 1004` va justo después de `MSG1`), porque si no el receptor lo descartaría por estar fuera de ventana y el experimento no probaría nada.

Cómo se resuelve de verdad. Si la aplicación necesita fronteras, **tiene que ponerlas ella misma**: un prefijo de longitud o un delimitador. Por eso HTTP tiene `Content-Length` y `Transfer-Encoding: chunked`, y por eso el RPC usa *marshalling* con longitud. El único transporte con noción nativa de mensaje es **SCTP**, y por eso WebRTC lo usa para sus canales de datos.

⚠️ Actualización a 2026

Sigue igual — de memoria: números de secuencia de **32 bits** que numeran **bytes**; **ACK acumulativo** (`ack` = próximo byte esperado); **timers por segmento**; Jacobson/Karels con α = 7/8, β = 1/4, `RTO = SRTT + 4·RTTVAR`, piso y techo, y **backoff exponencial**; **3 ACKs duplicados** = pérdida aislada y RTO = pérdida grave; **PSH es una sugerencia**.

Cambió:

- La especificación vigente es `RFC 9293` **(2022, *Internet Standard*)**, que consolida `RFC 793` + `RFC 879`, `2873`, `6093`, `6429`, `6528`, `6691`. Presentar `RFC 793` como "la especificación de TCP" en 2026 está desactualizado. **Lo que no cambió son los mecanismos de esta clase**: `RFC 9293` describe la misma fiabilidad, reorganizó la redacción y el estado normativo. `RFC 1122` sigue definiendo el comportamiento de los hosts.
- **SACK =** `RFC 2018` **(1996)**, *Standards Track*: es **comportamiento estándar** y está en todos los stacks. Cuando un enunciado pida el diagrama "a la antigua", aclaralo.
- **Timestamps + Window Scale =** `RFC 7323` **(2014)**, que **obsolesce** `RFC 1323`; **PAWS es obligatorio** si ambos extremos activaron timestamps.
- **Initial Window = 10 MSS (** `RFC 6928` **, 2013)**: el default en Linux, Windows y macOS. Los apuntes de 2011 dicen 1 MSS porque la `RFC 5681` lo decía así: **citá las dos cosas y seguí lo que pida el enunciado.**
- **QUIC (** `RFC 9002` **, 2021)** tiene **su propia recuperación de pérdida**: para HTTP/3 el flujo llega confirmado **por stream**, y el *Connection ID* permite sobrevivir al *NAT rebinding*, que en TCP se rompe.
- **MPTCPv1 (** `RFC 8684` **, 2020)** extiende la fiabilidad a varias rutas (cada subflujo aplica `RFC 9293` por separado). Desplegado pero con cuota marginal (~0,4 % de los bytes pico).
- ⚠️ **RACK** existe en stacks reales pero **no figura en** `ACTUALIZACIONES.md`: **verificar**.

> **Consecuencia para el examen:** los números de secuencia, el ACK acumulativo, los timers, Jacobson/Karels y la ventana deslizante **se preguntan exactamente igual que en 2011**. Lo único que cambia es la etiqueta: si citás "la RFC de TCP", es `RFC 9293`.

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — RTO con Jacobson/Karels a partir de 4 muestras

Datos. α = 7/8, β = 1/4. Cuatro muestras de RTT en orden: `R₁ = 100 ms`, `R₂ = 120 ms`, `R₃ = 90 ms`, `R₄ = 130 ms`. Sin piso ni techo activos.

Paso 1 — Inicializar con la primera muestra. Con un solo dato no hay histórico, así que no se puede calcular una desviación:

```
SRTT   = R₁ = 100 ms
RTTVAR = R₁ / 2 = 50 ms
RTO    = SRTT + 4·RTTVAR = 100 + 4·50 = 300 ms
```

Paso 2 — Actualizar con R_2 = 120 ms. *Orden importante:* primero `RTTVAR` (que usa el `SRTT` **anterior**), después `SRTT`.

```
RTTVAR = 0,75 · 50 + 0,25 · |100 − 120| = 37,5 + 0,25·20 = 37,5 + 5 = 42,5 ms
SRTT   = 0,125 · 100 + 0,875 · 120    = 12,5 + 105       = 117,5 ms
RTO    = 117,5 + 4·42,5 = 117,5 + 170 = 287,5 ms
```

Paso 3 — Actualizar con R_3 = 90 ms.

```
RTTVAR = 0,75 · 42,5 + 0,25 · |117,5 − 90| = 31,875 + 6,875 = 38,75 ms
SRTT   = 0,125 · 117,5 + 0,875 · 90       = 14,6875 + 78,75 = 93,4375 ms
RTO    = 93,4375 + 4·38,75 = 93,4375 + 155 = 248,44 ms
```

Paso 4 — Actualizar con R_4 = 130 ms.

```
RTTVAR = 0,75 · 38,75 + 0,25 · |93,4375 − 130| = 29,0625 + 9,1406 = 38,2031 ms
SRTT   = 0,125 · 93,4375 + 0,875 · 130       = 11,6797 + 113,75 = 125,43 ms
RTO    = 125,43 + 4·38,2031 = 125,43 + 152,81 = 278,24 ms
```

Interpretación — qué está pasando. El RTO **oscila entre 248 y 300 ms**: ni sigue al RTT puntualmente (porque `α = 7/8` pesa el pasado 7 veces más que el presente) ni se queda corto. Es exactamente lo que se busca: **un RTO que cambia suave y no reacciona a un solo RTT raro**. Si el RTO fuera "el último RTT + un margen", el valor saltaría de 90 a 130 ms según la última muestra, y con 90 ms de RTO real habrías retransmitido de más.

Paso 5 — Backoff exponencial. Si ahora se pierde un segmento y vence el RTO, los timeouts sucesivos dan `2 · 278,24 = 556,48 ms`, luego `1112,96`, luego `2225,92`. Y cuando llega un ACK que **no** corresponde a una retransmisión, se recalcula desde cero: el backoff solo aplica mientras no haya progreso.

⚠️ Errores típicos

- **Usar `α` y `β` al revés.** `α = 7/8` es para `SRTT`; `β = 1/4` para `RTTVAR`. **α grande → suavizo lento; β pequeño → reacción rápida.**
- **Olvidar la inicialización** `SRTT = R`, `RTTVAR = R/2`.
- **Confundir `RTT` con `RTO`.** El `RTT` es lo que **midió**; el `RTO`, lo que **va a esperar**.
- **Creer que el RTO se recalcula en cada ACK.** Se recalcula por **muestra de RTT** no ambigua.

2.2 Ejercicio 2 — BDP y tamaño mínimo de ventana

Datos (a–d). Enlace de **100 Mbps** (datos de capa 3, sin overhead de capas 1 y 2), RTT extremo a extremo **80 ms**, MTU 1500 bytes.

a) ¿Cuál es el BDP?

```
BDP = 100 · 10⁶ bits/s × 0,080 s = 8 · 10⁶ bits = 8·10⁶ / 8 = 1 000 000 bytes
    ≈ 1 MB   (en KiB: 1 000 000 / 1024 = 976,6 KiB)
```

b) ¿Cuántos segmentos de tamaño máximo hacen falta?

MSS = MTU − cabecera IP − cabecera TCP = 1500 − 20 − 20 = 1460 bytes

```
1 000 000 / 1460 = 684,93 → 685 segmentos
```

c) ¿Tamaño mínimo de ventana para usar toda la capacidad?

```
ventana_mínima = BDP = 1 000 000 bytes
comprobación: 1 000 000 / 0,080 s = 12 500 000 B/s = 100 · 10⁶ bits/s = 100 Mbps ✓
```

d) ¿Y con la máxima del campo `window` de 16 bits, sin *window scale*?

```
ventana = 65 535 bytes (2¹⁶ − 1)
throughput = 65 535 / 0,080 s = 819 187,5 B/s = 6,55 Mbit/s
```

Interpretación — y acá está la moraleja. Con el campo de 16 bits **a secas**, este enlace de 100 Mbps **es inalcanzable**: sacás 6,55 Mbit/s como máximo. **Es el ejemplo perfecto de por qué un protocolo de 1992 necesita revisión**: el campo de 16 bits bastaba para las redes de entonces.

⚠️ Errores típicos

- **Olvidar `/8`.** Si mezclás bits con bytes te da 8 MB en vez de 1 MB: factor 8 de error.
- **Usar 1000 en vez de 1024** (o al revés) sin decirlo. Declaralo.
- **Confundir "ventana mínima" con "ventana igual a un MSS".** El mínimo para **usar todo el enlace** es el BDP; el mínimo para **no bloquearse** es 1 MSS (o 2, por el *silly window syndrome*).
- **Olvidar la MTU.** MSS no es "1500": es `1500 − 20 − 20 = 1460` sin opciones, o `1500 − 20 − 32 = 1448` con timestamps.

2.3 Ejercicio 3 — Comparar cuatro ventanas sobre el mismo enlace

Datos. Enlace de **1 Gbps**, RTT **50 ms** uniforme. Archivo de **4 MB** (`4 · 10⁶` bytes).

```
BDP = 10⁹ · 0,050 / 8 = 5 · 10⁷ bits = 6 250 000 bytes ≈ 5,96 MiB   (4281 MSS)

throughput = min( ventana / 0,050 , 125 · 10⁶ B/s )
tiempo     = 4 · 10⁶ B / throughput
```

Ventana	`ventana/RTT`	Throughput real	Tiempo
8 KiB = 8 192 B	163 840 B/s	**1,31 Mbit/s** (limitado por ventana)	**24,4 s**
64 KiB = 65 536 B	1 310 720 B/s	**10,5 Mbit/s** (limitado por ventana)	**3,05 s**
763 KiB = 781 250 B	15 625 000 B/s	**125 Mbit/s** (ventana justo = enlace)	**0,256 s**
2 MiB = 2 097 152 B	41 943 040 B/s	**1 000 Mbit/s** (limitado por enlace)	**0,032 s**

Verificación de la última fila: `4·10⁶ / 41,94·10⁶ = 0,095 s`, **mayor** que los `0,032 s` que tarda el enlace: imposible, el enlace no puede ir más rápido que sí mismo. La ventana de 2 MiB "sobra".

Interpretación. De 8 KiB a 64 KiB el tiempo bajó de 24,4 s a 3,05 s: **8 veces menos tiempo por 8 veces más ventana**, porque ahí estás limitado por `ventana/RTT`. A partir de `ventana = BDP` el enlace se satura y **agrandar la ventana ya no sirve**: es el techo físico. Por eso definir bien `SO_RCVBUF` / `SO_SNDBUF` y el *window scale* es la diferencia entre 1,31 y 1000 Mbit/s — y por eso el `rwnd` es un problema de **flujo**: agrandarlo no le pide nada a la red.

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 Examen 2010-12, Pregunta 4 (8 puntos) — números de secuencia y timers

> **Fuente:** Examen `sirc1012` — 17 de diciembre de 2010. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1012.pdf **Texto del examen:**
>
> En el protocolo de capa de transporte TCP: a) ¿Por qué se necesitan números de secuencia? b) ¿Por qué son necesarios timers?

Respuesta oficial a) (textual):

> Para aprovechar mejor el enlace, TCP es también un protocolo **pipelined** por lo que el emisor envía varios paquetes antes de recibir confirmación por alguno de ellos. Por esta razón, el emisor debe poder distinguir **a qué paquete o paquetes corresponde una determinada confirmación**, por lo que numera los paquetes enviados secuencialmente.

Los 4 puntos: (1) **pipelining**: para aprovechar el enlace A manda varios antes de esperar; (2) **consecuencia**: hay confirmaciones en vuelo y son ambiguas sin identificador; (3) **solución**: numerar, y el ACK usa esos números; (4) **matiz que suma**: los números son **de bytes**, no de segmentos, y el ACK es **acumulativo**.

Por qué los números no son un lujo del pipelining. Aunque TCP fuera `stop-and-wait`, haría falta algo: si el receptor asienta y se pierde el ACK, hay que saber qué reenviar. El pipelining **amplifica** la necesidad, no la crea. Y si además hay **reordenamiento** (ida y vuelta por caminos distintos), sin números el receptor no podría ni ordenar.

Respuesta oficial b) (textual):

> De no recibir esta confirmación, el emisor no sabe si esto se debe a que se perdió el paquete enviado, la confirmación o simplemente alguno de los dos está demorado. **La única solución es retransmitir el paquete no confirmado.** Para esta retransmisión debe esperar un tiempo prudencial luego de enviado cada paquete, por lo que necesita un timer que le indique cuándo hacerlo.

Los 4 puntos: (1) **las tres causas indistinguibles**: se perdió el dato / se perdió el ACK / tardó; (2) **por qué obliga a retransmitir**: no hay forma de distinguirlas, y no retransmitir rompe la fiabilidad; (3) **por qué además hace falta saber *cuándo***: esperar un tiempo prudencial; (4) **un timer por segmento**, no por conexión.

Si querés ir más allá (suma): el RTO no es fijo sino una estimación (Jacobson/Karels, §1.4), ante timeout se **duplica** (*backoff exponencial*), y con timestamps (`RFC 7323`) el problema de *Karn* se resuelve.

3.2 Examen 2009-12, Pregunta 2 (10 puntos) — fiabilidad en transporte vs. en enlace

> **Fuente:** Examen `sirc0912` — 12 de diciembre de 2009. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc0912.pdf **Texto del examen:**
> Se considera un servicio de transmisión confiable. a) Señale las diferencias de implementación en capa de transporte vs. capa de enlace. b) En una red de paquetes arbitraria, ¿es posible implementar un servicio de transmisión confiable en capa de enlace? Justifique su respuesta.

Respuesta oficial a): "En capa de transporte se usan los servicios de la capa de red y la señalización del protocolo de transporte, que implica **realimentación de extremo a extremo**, teniendo en cuenta la carga de red (congestión). En capa de enlace se señaliza en base a la **capacidad del enlace**."

Respuesta oficial b): "**No es posible** porque pueden existir múltiples hops, con capas de enlace diferentes."

	Capa de enlace	**Capa de transporte (TCP)**
Alcance de la fiabilidad	**por salto** (*per hop*)	**extremo a extremo**
Qué señaliza	capacidad del **enlace**	carga de la **red**
Estado que guarda	1 paquete a la vez (2 con Go-Back-N)	una **ventana** de N bytes
Numeración	por **trama** del enlace	por **bytes**, global a la conexión
Detecta pérdida	por **CRC** del enlace	por **ACK + timer**, y **no sabe** quién falló
Congestión	no la maneja	**sí** (es su razón de ser)
Depende de que los saltos sean fiables	**sí, crítico**	**no, en absoluto**

La parte b) tiene que ir más allá de "hay varios hops". El contraejemplo mínimo: el paquete se pierde **dentro de R2**. La capa 2 del enlace de entrada a R2 no ve nada raro (entró bien y salió bien), la del enlace de salida ve un paquete entrar y lo entrega sin retransmitir nunca, y la capa 3 descartó el paquete sin avisar a nadie. **El paquete se perdió PARA SIEMPRE**: ningún mecanismo de capa 2 lo detecta.

Y una razón estructural más fuerte: el número de secuencia de capa 2 no atraviesa los routers. Para que una capa de enlace fuese *extremo a extremo* tendría que: **(1)** mantener estado por flujo en cada router (o sea, reimplementar TCP en cada salto); **(2)** usar números de secuencia globales, imposibles si cada capa 2 no sabe nada de lo que hay aguas arriba; **(3)** mandar ACKs de extremo a extremo, y entonces ya no es capa de enlace, es transporte con otro nombre.

Conclusión (una frase): la fiabilidad **extremo a extremo** es la única que sobrevive a una red no confiable. Por eso está **arriba** y no abajo.

3.3 Examen 2001-12, Problema 2 — el flag PUSH

Resuelto en §1.8. Lo que hay que dejar asentado:

- **Tesis:** PSH no define fronteras de mensaje; es un bit de sugerencia.
- **Prueba de laboratorio:** un bridge con buffer de retardo reordena y/o segmenta.
- **Los dos escenarios pedidos:** (a) dos mensajes que llegan juntos al `read()`; (b) un mensaje entero y parte del siguiente.
- **La solución real:** fronteras a nivel de aplicación (`Content-Length`, *marshalling* con longitud).

PARTE 4 — CIERRE

📌 Resumen en 10 líneas

1. **Ordenada** → números de secuencia · **sin duplicados** → el receptor descarta · **sin errores** → checksum + retransmisión.
2. TCP es **pipelined**: manda varios segmentos antes de tener ACKs, y por eso el ACK tiene que decir **cuál** confirma.
3. Los números son **de bytes**; el `ack` es el **próximo byte esperado**, y es **acumulativo**.
4. **Timer por segmento**: sin ACK hay tres causas **indistinguibles** y en las tres "no sé".
5. La única salida es **retransmitir**, y para eso hay que saber **cuándo** → RTO.
6. **Jacobson/Karels**: `SRTT = α·SRTT + (1−α)·R` con α = 7/8; `RTTVAR` con β = 1/4; **`RTO = SRTT + 4·RTTVAR`**.
7. Ante timeout el RTO **se duplica** (*backoff exponencial*).
8. **Ventana deslizante** = bytes sin confirmar; para llenar un enlace hace falta **`ventana ≥ BDP = ancho_de_banda × RTT`**.
9. **Fast retransmit** (3 dupACKs) recupera rápido; **SACK (`RFC 2018`)** dice *qué* bloques llegaron, así que recupera en 1 RTT en vez de varios.
10. **PSH no garantiza un mensaje por `read()`**: TCP es un flujo de bytes y las fronteras las pone la aplicación.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. El `ack` de un segmento es 1120. ¿Cuántos bytes recibió el receptor como mínimo?

<details>
<summary>Respuesta</summary>
</details>

2. A mandó 5 segmentos y el RTO del **primero** se venció. ¿Qué tres cosas pueden estar pasando? ¿Puede TCP saber cuál es?

<details>
<summary>Respuesta</summary>
</details>

3. Calculá el RTO con Jacobson/Karels si `SRTT = 100 ms`, `RTTVAR = 20 ms` y llega `R = 100 ms`.

<details>
<summary>Respuesta</summary>
</details>

4. Ancho de banda 1 Gbps, RTT 20 ms. ¿Qué ventana mínima para saturarlo?

<details>
<summary>Respuesta</summary>
</details>

5. ¿Por qué `SRTT + 4·RTTVAR` y no `SRTT · 2`?

▶ Respuesta

6. Un switch en el medio reordena dos segmentos con PSH: pasa primero el segundo. ¿Qué lee la aplicación en el destino?

▶ Respuesta

7. ¿Qué cambia, en 2026, respecto de lo que se enseñaba en 2011?

▶ Respuesta

8. ¿Por qué no se puede medir el RTT de un segmento retransmitido?

▶ Respuesta

🔗 Referencias

- **Examen** `sirc1103` — P4 (10 pts): 4 segmentos con y sin pérdida · Problema 1 (30 pts): ECN
- **Examen** `sirc1012` — P4 (8 pts): números de secuencia y timers
- **Examen** `sirc0912` — P2 (10 pts): fiabilidad en transporte vs. enlace
- **Examen** `sirc0112` — Problema 2: el flag PUSH y la segmentación
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 6 §6.5 (pág. ~590)
- `RFC 9293` — TCP (2022, *Internet Standard*) · `RFC 2018` — SACK (1996)
- `RFC 7323` — Timestamps, Window Scale y PAWS (2014) · `RFC 5681` — Control de congestión (2009)
- `RFC 6928` — IW = 10 MSS (2013) · `RFC 9002` — QUIC (2021) · `RFC 8684` — MPTCPv1 (2020)
- **Clase siguiente:** 20 — Control de flujo vs. control de congestión